Innominatum CONSULTORIA

Contentores e Virtualização

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Objetivo

Compreender o problema que os contentores resolvem, dominar os conceitos fundamentais do Docker e ser capaz de construir e executar uma aplicação Angular dentro de um contentor usando um Dockerfile multi-stage.

Objetivos Específicos

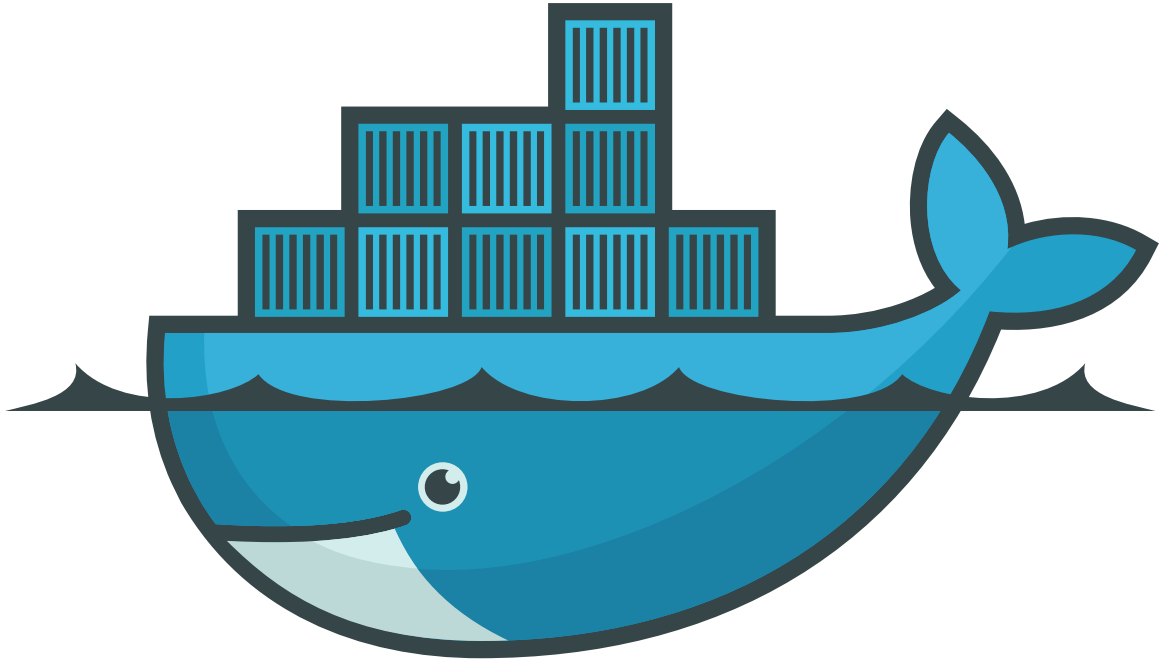
Ao final da sessão, os formandos deverão ser capazes de:

- Distinguir virtualização tradicional (VMs) de contentores;
- Dominar os conceitos: imagem, contentor, layer (camada), Dockerfile e registry;
- Construir um Dockerfile multi-stage para uma aplicação Angular;
- Usar os comandos essenciais do Docker e publicar imagens no Docker Hub ou GHCR.

DOCKER – O que é?

Docker é uma **plataforma** que permite criar, implantar e executar aplicações em contentores. Um contentor é um **ambiente isolado** que agrupa todas as dependências necessárias para executar uma aplicação, garantindo que ela funcione de maneira consistente em diferentes ambientes.

DOCKER



docker

Innominatum CONSULTORIA
IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

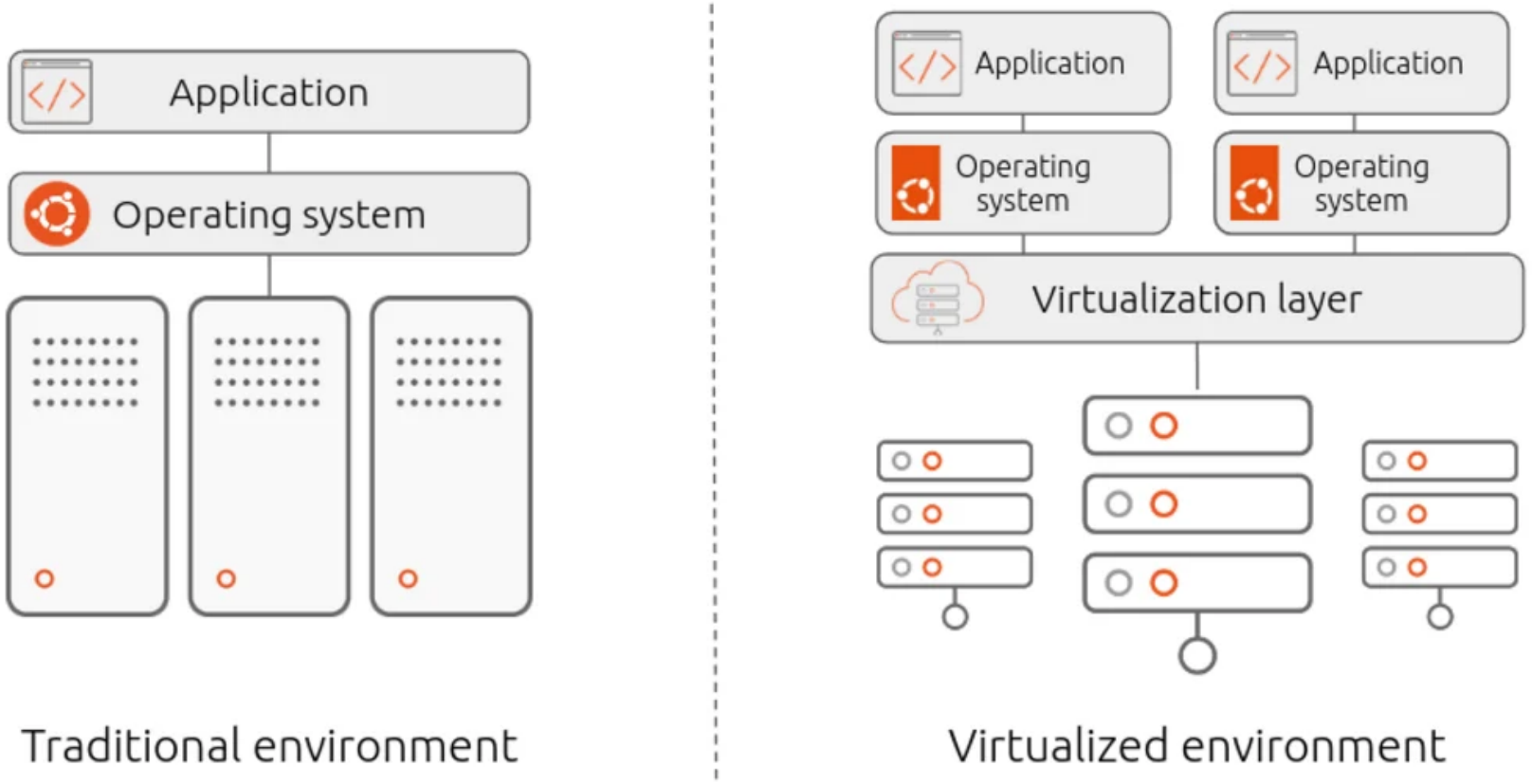
Innominatum CONSULTORIA

Virtualização

Virtualização tradicional (VMs) vs Contentores: diferenças e analogias práticas

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Virtualização



Definição

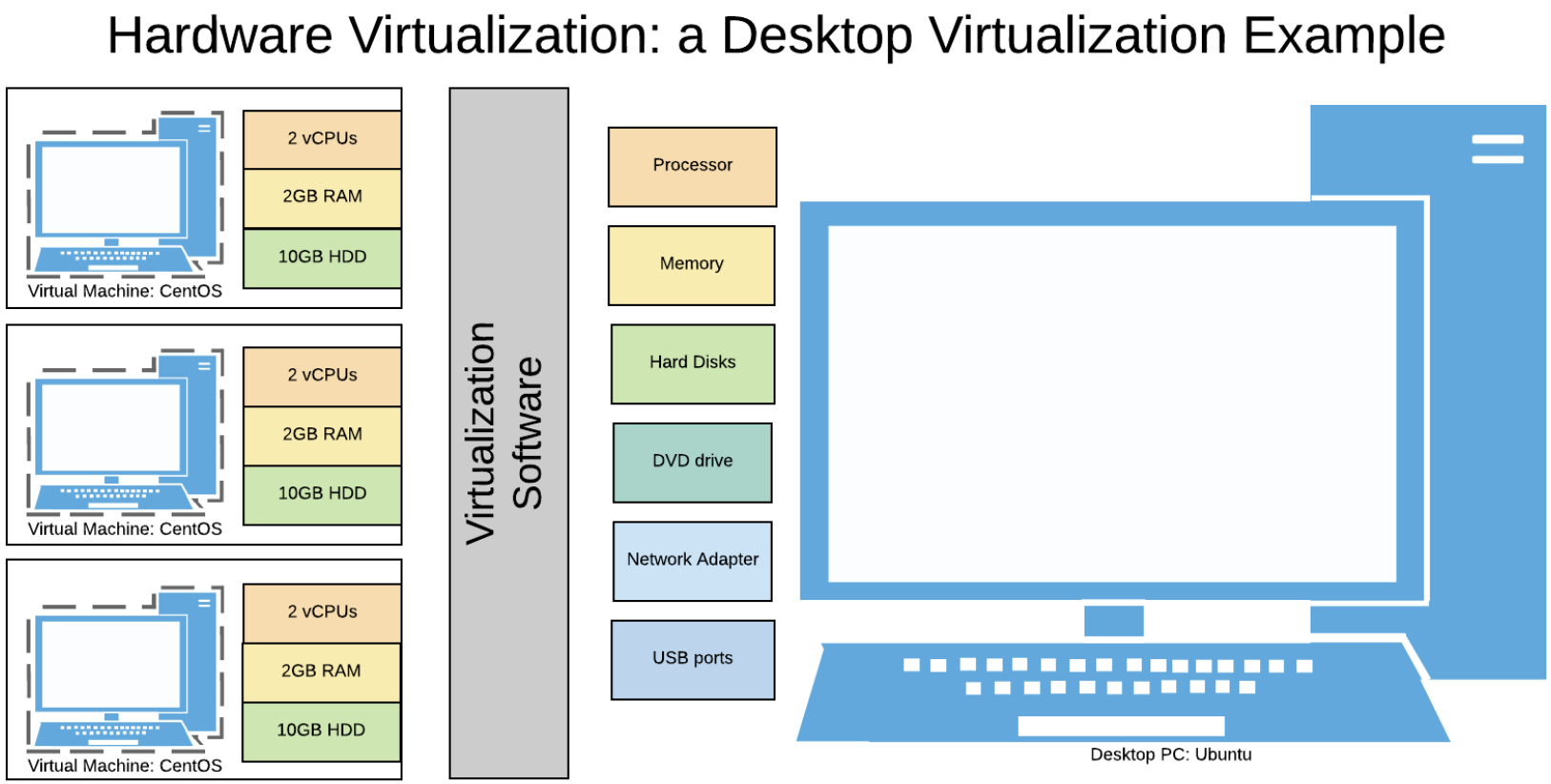
A **virtualização** é uma técnica que cria uma **versão virtual (uma “ilusão”) de recursos informáticos**, como um sistema operativo, armazenamento, rede ou computador inteiro.

Isso permite que **dispositivos, aplicações e utilizadores interajam com esses recursos como se fossem físicos**, mesmo que estejam a ser simulados por software.

Definição

- A virtualização permite um melhor uso dos recursos físicos disponíveis.
- Em vez de um hardware servir apenas um sistema ou aplicação, pode ser dividido e usado por vários ambientes virtuais ao mesmo tempo.

Definição


Hardware Virtualization: a Desktop Virtualization Example
2 vCPUs
2GB RAM
10GB HDD
Virtual Machine: CentOS
2 vCPUs
2GB RAM
10GB HDD
Virtual Machine: CentOS
2 vCPUs
2GB RAM
10GB HDD
Virtual Machine: CentOS
Virtualization Software
Processor
Memory
Hard Disks
DVD drive
Network Adapter
USB ports
Desktop PC: Ubuntu

O Problema

Um dos problemas mais frustrantes no desenvolvimento de software: o código corre perfeitamente no computador do programador mas falha noutro ambiente.

"Funciona na Minha Máquina"

O Problema

A causa é quase sempre a mesma: dependências de sistema diferentes. Bibliotecas de sistema operativo, variáveis de ambiente, configurações locais.

A solução ideal: Simular o ambiente de produção, de modo a que a aplicação possa ser desenvolvida num ambiente o mais próximo possível do ambiente de produção.

Innominatum CONSULTORIA

VMs vs Contentores

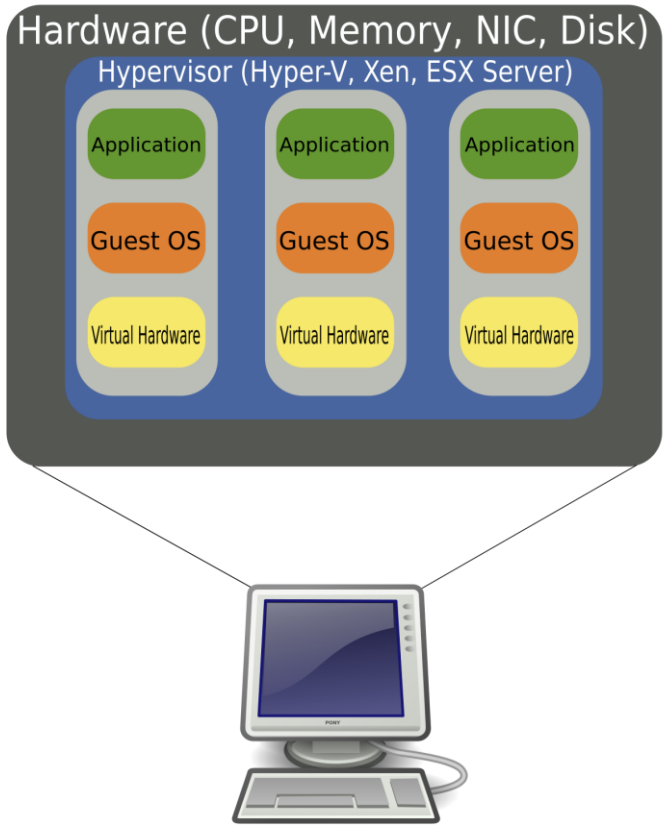
Duas abordagens de virtualização

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Virtual Machine

Uma Virtual Machine (VM) emula um computador completo: hardware virtual, BIOS, sistema operativo próprio. É como ter um computador dentro do computador.

Virtual Machine



Virtual Machine

Cada VM tem o seu próprio kernel, sistema de ficheiros, drivers e sistema operativo completo. Um servidor físico pode correr várias VMs em simultâneo.

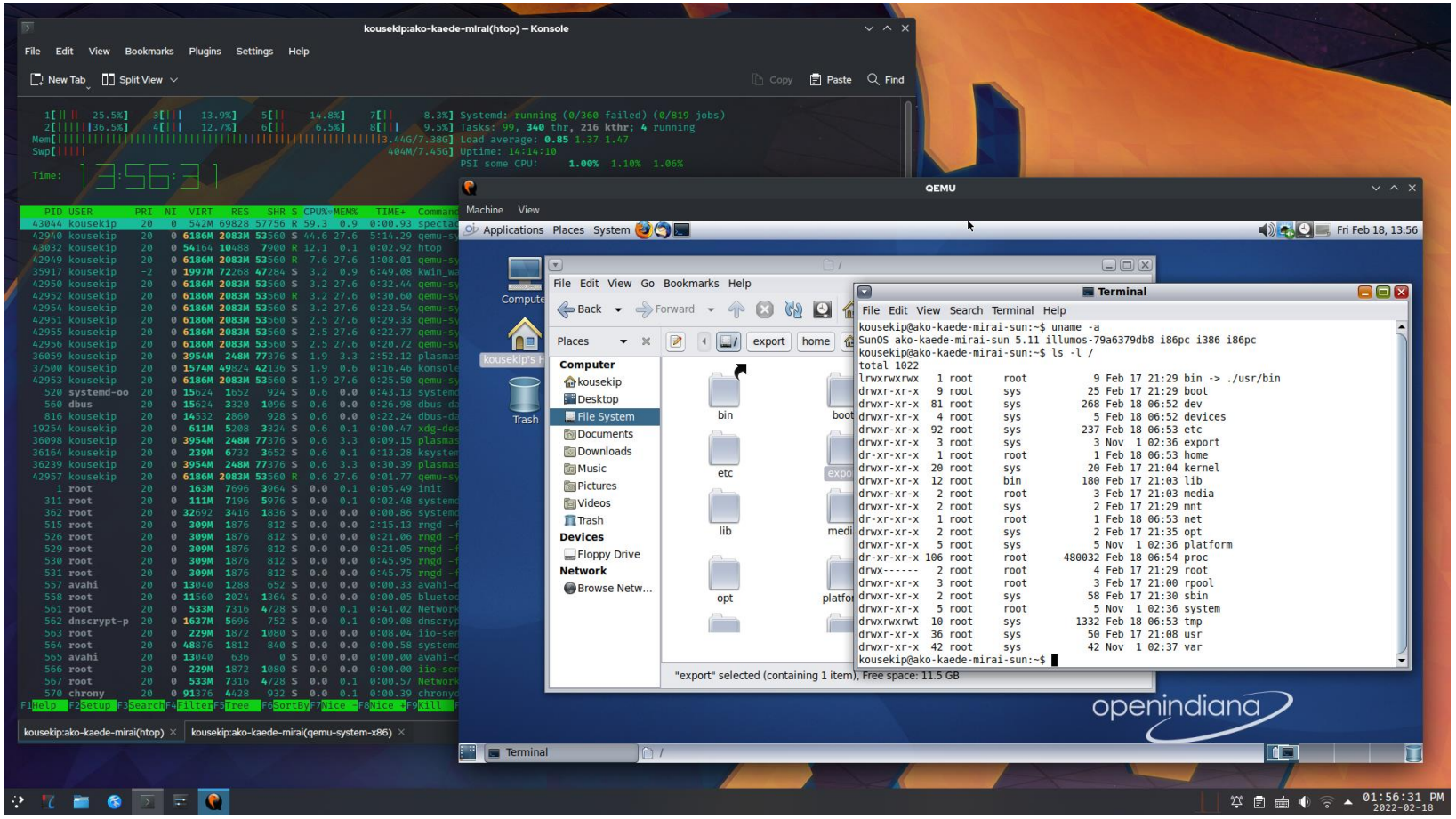
- **Vantagens**: isolamento total, cada VM é completamente independente;
- **Desvantagens**: muito pesadas (GBs de espaço), arranque lento (minutos), elevado consumo de recursos.

Virtual Machine

Casos de uso:

Quando se precisa de sistemas operativos diferentes no mesmo servidor, ou de isolamento total por razões de segurança.

Virtual Machine

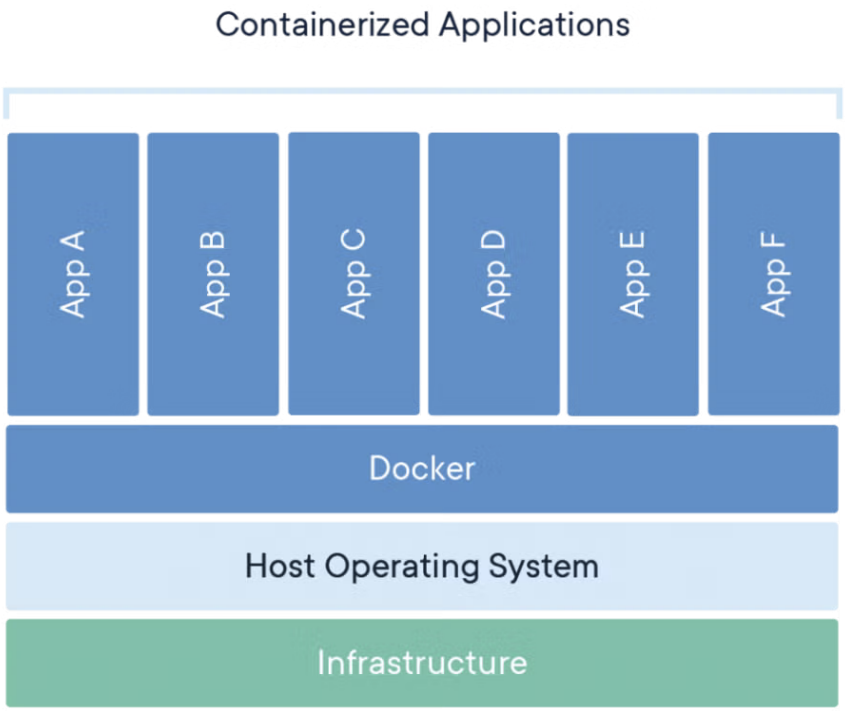


Innominatum CONSULTORIA
IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Contentores

Um contentor não virtualiza o hardware. Partilha o kernel do sistema operativo anfitrião mas isola o processo e as suas dependências numa cápsula própria.

Contentores



Contentores

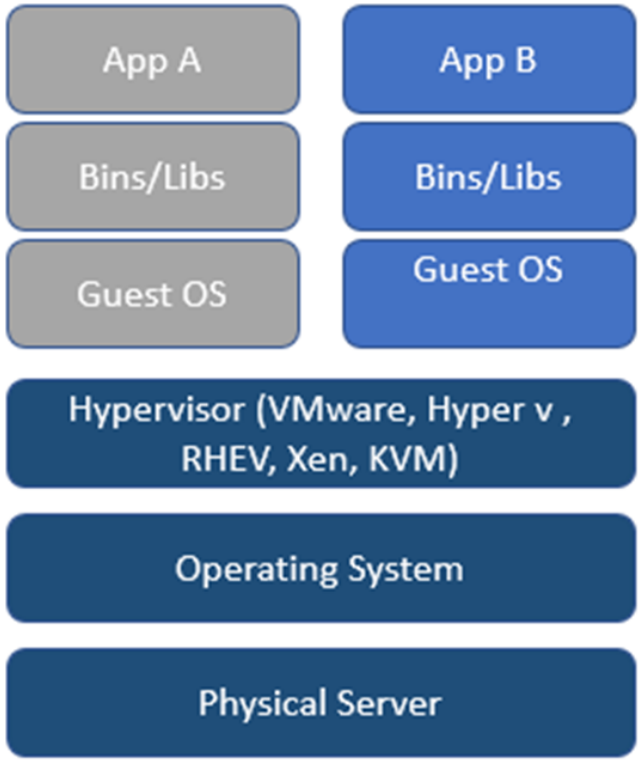
Cada contentor tem o seu sistema de ficheiros, variáveis de ambiente e bibliotecas, mas usa o mesmo kernel que o sistema anfitrião.

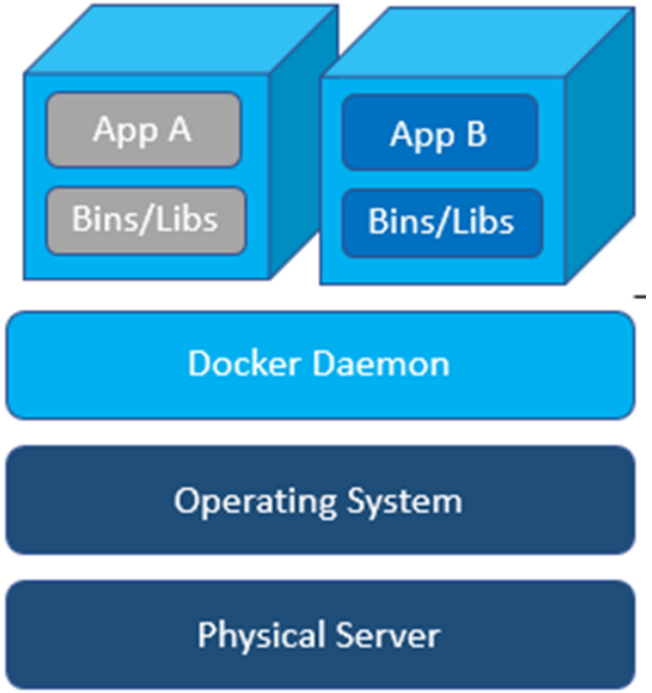
- **Vantagens**: extremamente leves (MBs), arranque instantâneo (segundos), eficientes em recursos. Uma máquina pode correr dezenas de contentores simultâneos.
- **Desvantagens**: dependência do kernel do sistema anfitrião (mesmo SO); e dados efémeros por omissão (persistência requer configuração explícita de volumes).

VMs vs Contentores

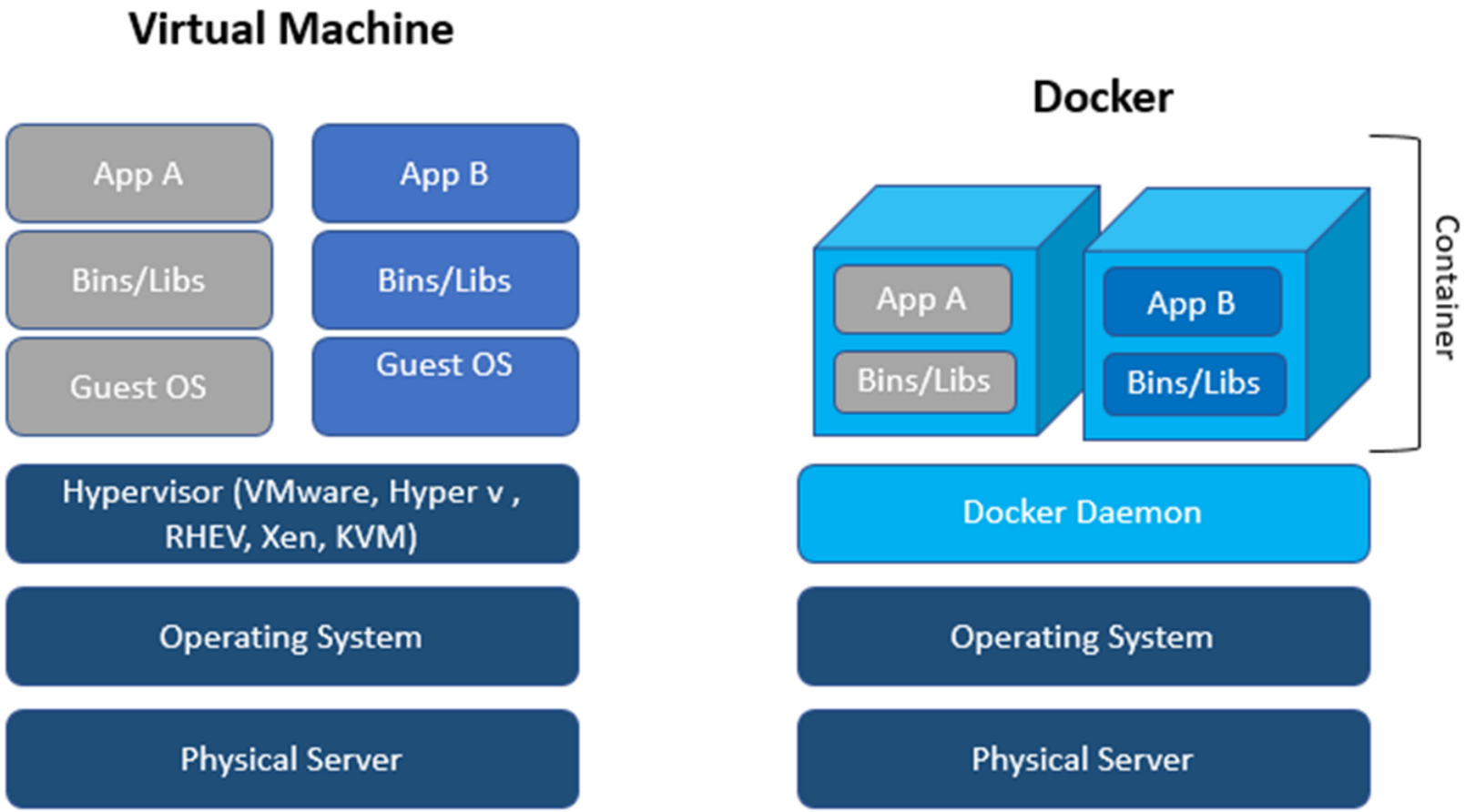
Uma VM é uma casa completa com fundações próprias. Um contentor é um apartamento num edifício partilhado com infraestrutura comum.

VMs vs Contentores





VMs vs Contentores



VMs vs Contentores

	Máquina Virtual (VM)	Contentor
Tamanho	GBs (inclui sistema operativo completo)	MBs (apenas dependências da aplicação)
Arranque	Minutos	Segundos ou menos
Isolamento	Kernel próprio, isolamento total	Kernel partilhado, isolamento de processo
Portabilidade	Imagem pesada, difícil de distribuir	Imagem leve, fácil de partilhar
Casos de uso	Segurança máxima ou execução de SOs diferentes	Microserviços e deploy rápido

VMs vs Contentores

Pergunta:

Porquê usar uma máquina virtual?

Innominatum CONSULTORIA

Conceitos Fundamentais em Docker

Imagem, contentor, layer, Dockerfile, registry

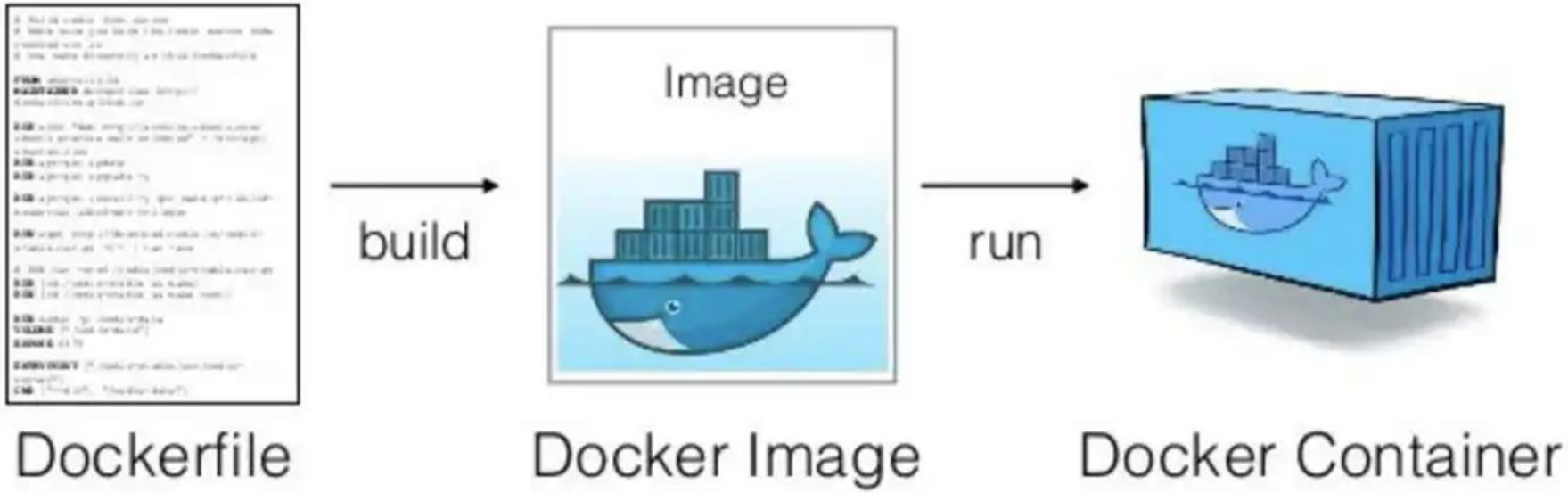
IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Conceitos-Chave

Antes de usar o Docker, é essencial perceber estes conceitos:

- **Imagem**: modelo imutável que descreve o sistema de ficheiros e configuração da aplicação. É como uma forma.
- **Contentor**: instância em execução de uma imagem. Pode haver múltiplos contentores do mesmo molde.
- **Layer**: cada instrução no Dockerfile cria uma camada. As camadas são armazenadas em cache e reutilizáveis entre imagens.
- **Dockerfile**: ficheiro de texto com as instruções para construir uma imagem passo a passo.
- **Registry**: repositório de imagens. Docker Hub é o público. GHCR (GitHub Container Registry) é o da GitHub.

Conceitos



Dockerfile

Descrever o ambiente como código

Anatomia de um Dockerfile

Um Dockerfile é uma sequência de instruções que o Docker executa para construir uma imagem. Cada instrução cria uma nova layer.

Boas práticas: usar imagens alpine (mais leves), colocar instruções que mudam menos no início (melhor cache), e não correr como root.

Anatomia de um Dockerfile

- **FROM**: define a imagem base (ex: node:20-alpine).
- **WORKDIR**: define o diretório de trabalho dentro do contentor;
- **COPY**: copia ficheiros do host para o contentor;
- **RUN**: executa comandos durante o build (ex: npm install);
- **EXPOSE**: declara a porta que o processo vai usar (documentação);
- **CMD**: o comando que corre quando o contentor arranca.

Anatomia de um Dockerfile

```
# Usando uma imagem base do Node
FROM node:14

# Criando diretório de trabalho
WORKDIR /usr/src/app

# Copiando arquivos de dependências
COPY package*.json ./

# Instalando dependências
RUN npm install

# Copiando o restante do código
COPY . .

# Expondo a porta da aplicação
EXPOSE 3000

# Comando para rodar a aplicação
CMD [ "node", "app.js" ]
```

Multi-Stage Build

Uma aplicação Angular precisa do Node.js para compilar, mas não precisa dele para servir.

O multi-stage build aproveita isso.

Multi-Stage Build

- **Stage 1 (build):** FROM node:20-alpine. Instala dependências com npm ci, corre ng build. Resultado: a pasta dist/ com os ficheiros estáticos.
- **Stage 2 (serve):** FROM nginx:alpine. Copia apenas o dist/ do stage anterior. Expõe a porta 80. A imagem final não contém Node.js. Resultado: imagem de produção muito mais pequena (nginx:alpine ~ 20MB vs node:20-alpine ~ 180MB). Superfície de ataque mínima.

Comandos Essenciais Docker

Os comandos mais usados no dia a dia:

- **Construir**: docker build -t nome:tag . (lê o Dockerfile da diretoria atual e cria a imagem);
- **Correr**: docker run -p 8080:80 nome (inicia um contentor e mapeia a porta 8080 do host para a 80 do contentor);
- **Inspecionar**: docker ps (contentores ativos) | docker logs id (ver output) | docker inspect id (detalhes);
- **Parar e limpar**: docker stop id | docker rm id (apagar contentor) | docker rmi nome (apagar imagem).

Boas Práticas e Registos

.dockerignore, Docker Hub e GHCR

.dockerignore

- exclui ficheiros do contexto de build (como .gitignore).
- Excluir sempre: node_modules/, dist/, .git/, .env

Sem .dockerignore o Docker copia tudo para o *daemon*, incluindo node_modules (centenas de MBs) antes de sequer começar a construir.

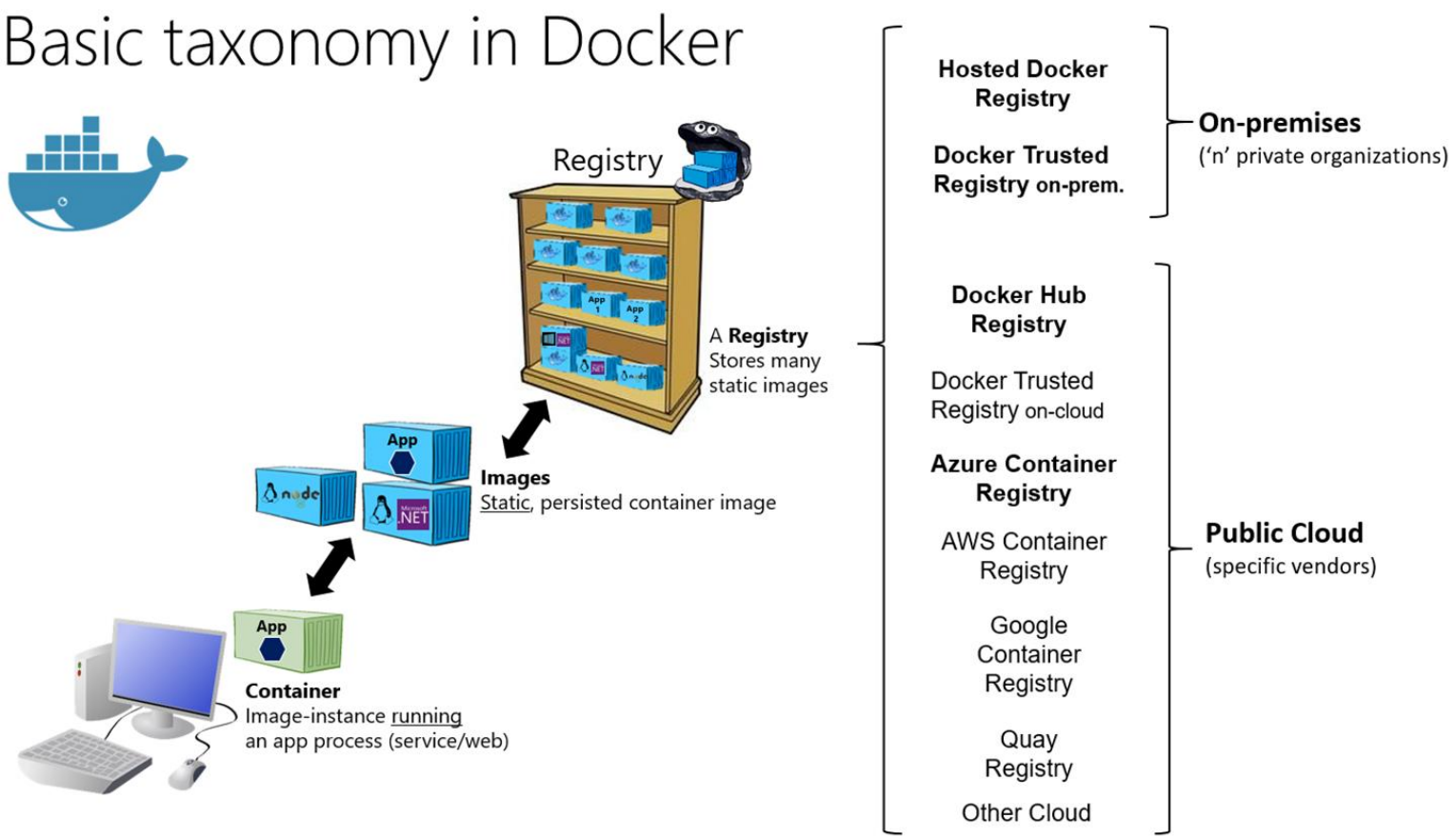
.dockerignore

```
# Ignore node_modules
node_modules/

# Ignore log files
logs/
*.log

# Ignore environment files
.env

# Ignore Dockerfile
Dockerfile
```

Docker Hub

Os desenvolvedores devem armazenar imagens num **Registo**, que funciona como uma biblioteca de imagens e é necessário ao implantar em orquestradores de produção. O Docker mantém um Registo público por meio do **DockerHub**. Como alternativa, as empresas podem ter um Registo privado local para as suas próprias imagens do Docker.

Docker Hub


Basic taxonomy in Docker
Registry
App 1
App 2
A Registry
Stores many
static images
App
Images
Static, persisted container image
App
Container
Image-instance running
an app process (service/web)
Hosted Docker Registry
Docker Trusted Registry on-prem.
On-premises
('n' private organizations)
Docker Hub Registry
Docker Trusted Registry on-cloud
Azure Container Registry
AWS Container Registry
Google Container Registry
Quay Registry
Other Cloud
Public Cloud
(specific vendors)

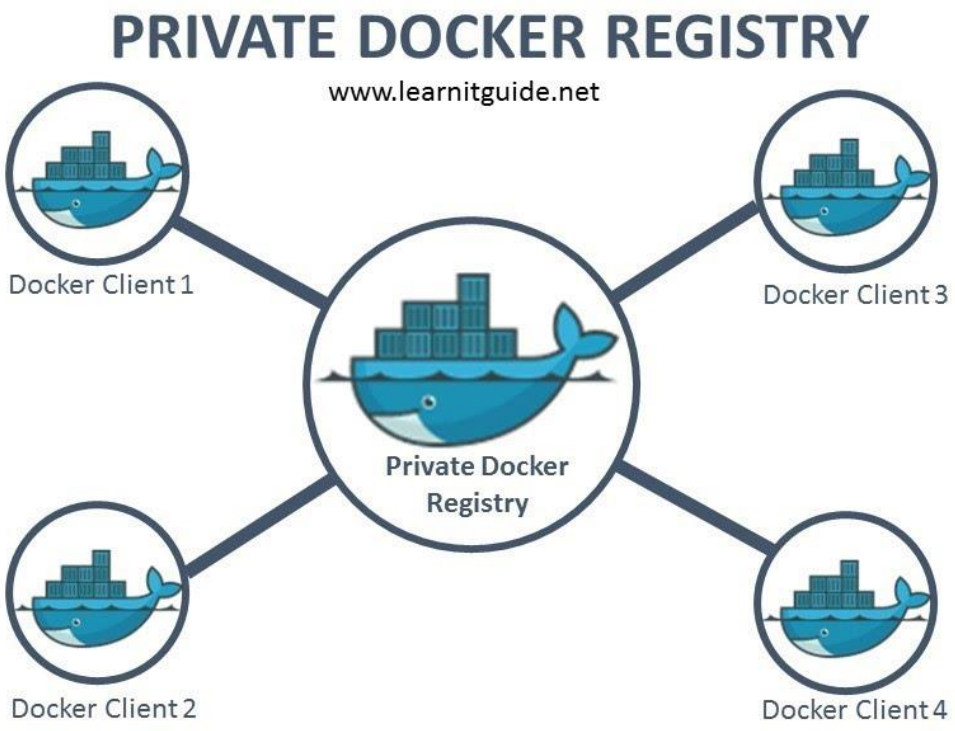
Docker Hub

Registo público e gratuito.

docker push utilizador/imagem:tag para publicar.

Qualquer pessoa pode fazer pull.

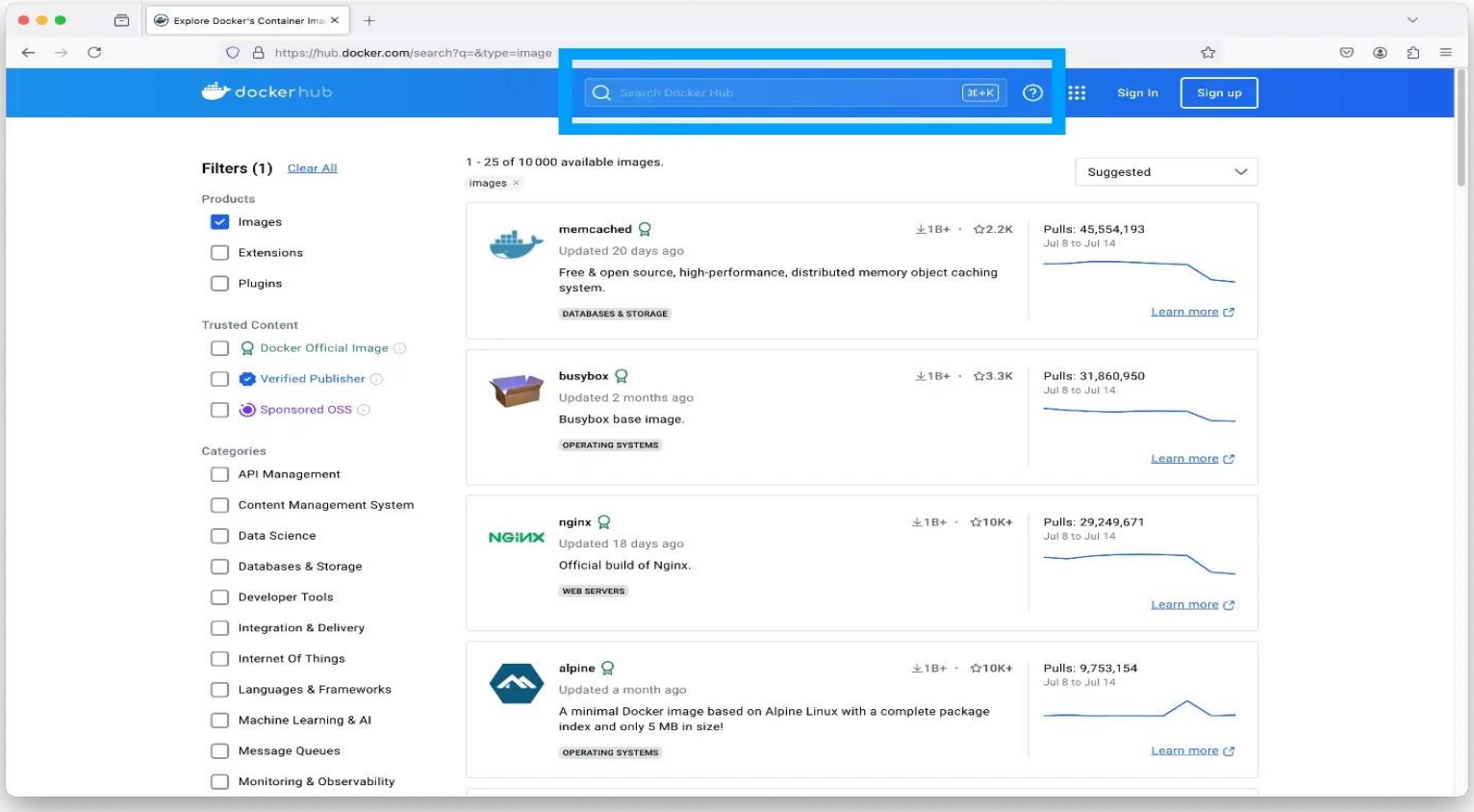
Docker Hub


PRIVATE DOCKER REGISTRY
www.learnitguide.net
Docker Client 1
Docker Client 3
Private Docker Registry
Docker Client 2
Docker Client 4

Docker Hub

dockerhub
The world's largest container registry
docker hub
Explore
Repositories
Organizations
Help
MD
mdock
Docker / Repositories / mdock / General
General
Tags
Builds
Activity
Settings
mdock / my-app
Description
This is my first app on Docker Hub!
Last pushed: an hour ago
Docker commands
To push a new tag to this repository.
Public view
docker push mdock/my-app:tagname
Tags
Automated Builds

Docker Hub


Explore Docker's Container Ima
https://hub.docker.com/search?q=&type=image
dockerhub
Search Docker Hub
⌘+K
Sign In
Sign up
Filters (1)
Clear All
Products
Images
Extensions
Plugins
Trusted Content
Docker Official Image
Verified Publisher
Sponsored OSS
Categories
API Management
Content Management System
Data Science
Databases & Storage
Developer Tools
Integration & Delivery
Internet Of Things
Languages & Frameworks
Machine Learning & AI
Message Queues
Monitoring & Observability
1 - 25 of 10 000 available images.
images
Suggested
memcached
Updated 20 days ago
Free & open source, high-performance, distributed memory object caching system.
DATABASES & STORAGE
1B+ · 2.2K
Pulls: 45,554,193
Jul 8 to Jul 14
Learn more
busybox
Updated 2 months ago
Busybox base image.
OPERATING SYSTEMS
1B+ · 3.3K
Pulls: 31,860,950
Jul 8 to Jul 14
Learn more
nginx
Updated 18 days ago
Official build of Nginx.
WEB SERVERS
1B+ · 10K+
Pulls: 29,249,671
Jul 8 to Jul 14
Learn more
alpine
Updated a month ago
A minimal Docker image based on Alpine Linux with a complete package index and only 5 MB in size!
OPERATING SYSTEMS
1B+ · 10K+
Pulls: 9,753,154
Jul 8 to Jul 14
Learn more

GHCR (GitHub Container Registry)

Registo integrado no GitHub. Imagens privadas gratuitas.

URL: ghcr.io/utilizador/imagem:tag

Boas práticas de tagging: usar tags semânticas (v1.2.3), latest para a última estável, e sha do commit para rastreabilidade.

Fluxo Completo: do Código ao Contentor

O fluxo Docker na prática resume-se a quatro passos que se repetem em cada deploy.

- **Passo 1**: escrever o Dockerfile na raiz do projeto. Multi-stage se possível. Criar o .dockerignore.
- **Passo 2**: docker build -t nome:v1 . - lê o Dockerfile e constrói a imagem localmente. Verificar com docker images.

Fluxo Completo: do Código ao Contentor

- **Passo 3**: `docker run -p 8080:80 nome:v1` - executa a aplicação. Abrir localhost:8080 no browser para verificar.
- **Passo 4**: `docker push nome:v1` - publica a imagem no registry. A partir daqui qualquer servidor pode fazer pull e executar exatamente o mesmo ambiente.

Utilidades

- **Autenticar:** docker login
- **Alterar a tag:** docker tag nome-imagem username/nome-imagem:v1
- **Enviar para o hub:** docker push username/nome-imagem:v2

Innominatum CONSULTORIA

Contentores e Virtualização

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL